🧩 1. Optional Class (Java 8+)

✅ What is Optional?

Optional<T> is a container object that **may or may not contain a non-null value**.
It's used to **avoid NullPointerException** and make your code more expressive about the *possibility of nulls*.

✅ Why use Optional

- To represent "value might be absent" cleanly.
- To avoid explicit null checks (if (obj != null) everywhere).
- To encourage functional-style code using methods like map(), filter(), orElse() etc.

🧠 Common Factory Methods

Method	Description	Example	Result
Optional.of(value)	Creates an Optional with a **non-null** value. Throws NullPointerException if value is null.	Optional.of("Java")	✅ Optional[Java]
Optional.ofNullable(value)	Creates an Optional that can hold a **null** value (empty if null).	Optional.ofNullable(null)	⚠️ Optional.empty()
Optional.empty()	Creates an empty Optional.	Optional.empty()	⚪ Optional.empty()

⚖️ Difference between of() and ofNullable()

Feature	Optional.of()	Optional.ofNullable()
Accepts null?	❌ Throws NullPointerException	✅ Safe with null

Feature	Optional.of()	Optional.ofNullable()
When to use	When value **guaranteed non-null**	When value **may be null**
Example	Optional.of("Hello")	Optional.ofNullable(user.getName())

🧩 Example Usage

```
Optional<String> name = Optional.ofNullable(getUserName());


String finalName = name
    .filter(n -> n.length() > 3)
    .map(String::toUpperCase)
    .orElse("DEFAULT");


System.out.println(finalName);
```

- If getUserName() returns null → prints "DEFAULT".
- Otherwise, it transforms and returns uppercase if valid.

💡 Tip for interviews:

Use Optional as **return type** — not for fields or parameters.

🧱 2. SOLID Principles (Focus: Liskov Substitution Principle)

✅ What are SOLID principles?

They are 5 design principles to make code **maintainable, extensible, and flexible**:

Principle	Meaning
S – Single Responsibility	A class should have only one reason to change.
O – Open/Closed	Open for extension, closed for modification.

Principle	Meaning
L – Liskov Substitution	Subclasses must be usable through their base class without breaking behavior.
I – Interface Segregation	Many small, specific interfaces > one large interface.
D – Dependency Inversion	Depend on abstractions, not concrete classes.

🧩 Liskov Substitution Principle (LSP)

Definition:
If class B is a subclass of class A, then objects of A should be replaceable with objects of B without altering the correctness of the program.

In simple terms — **a subclass should behave like its parent class without surprises**.

🚫 Bad Example (Violation of LSP)

```
class Bird {
  void fly() {
    System.out.println("Flying...");
  }
}


class Ostrich extends Bird {
  void fly() {
    throw new UnsupportedOperationException("Ostrich can't fly!");
  }
}
```

Problem:
Ostrich *is* a Bird, but substituting Bird with Ostrich breaks the program because the behavior changes.

✅ **Good Example (Follows LSP)**

```
interface Bird {
   void eat();
}

interface Flyable {
   void fly();
}

class Sparrow implements Bird, Flyable {
   public void eat() { System.out.println("Eating"); }
   public void fly() { System.out.println("Flying"); }
}

class Ostrich implements Bird {
   public void eat() { System.out.println("Eating"); }
}
```

Here we separated behaviors correctly — no class breaks expectations.

💡 **Key Takeaway**

- Subclasses should **not weaken preconditions** or **strengthen postconditions**.
- Always ensure the **"is-a" relationship** makes logical sense.
- Prefer **composition or interface segregation** when a subclass can't fully support parent behavior.

🧬 **3. Design Pattern – Prototype Pattern**

✅ **Intent**

The **Prototype pattern** allows you to **clone existing objects** instead of creating new ones from scratch, which can be costly.

🧠 When to Use

- Object creation is **expensive or complex** (e.g., deep object graphs, database fetching).
- You want to avoid reinitializing large objects.
- You want to **clone objects** with slightly different states.

🧩 Structure

- **Prototype Interface:** declares clone() method.
- **Concrete Prototype:** implements cloning logic.
- **Client:** uses the prototype to create new instances.

🧰 Example

```
class Employee implements Cloneable {
  private int id;
  private String name;

  public Employee(int id, String name) {
    this.id = id;
    this.name = name;
  }

  @Override
  public Employee clone() {
    try {
      return (Employee) super.clone(); // shallow copy
    } catch (CloneNotSupportedException e) {
```

```
            throw new RuntimeException(e);
        }
    }

    public String toString() {
        return id + " - " + name;
    }
}
```

Usage:

```
public class PrototypeDemo {
    public static void main(String[] args) {
        Employee emp1 = new Employee(101, "John");
        Employee emp2 = emp1.clone();

        System.out.println(emp1); // 101 - John
        System.out.println(emp2); // 101 - John
    }
}
```

⚙️ Advantages

- Saves costly object creation time.
- Hides complex construction logic.
- Simplifies object creation for dynamic types.

⚠️ Disadvantages

- Cloning complex objects with references can cause **shallow copy issues**.
- Deep cloning requires custom logic (manual clone of nested objects).

💡 Real-world Example

- Creating GUI elements, shapes, or database entity templates.
- Spring Framework's **Bean scopes** internally leverage prototype concepts.

🧩 **Interview Tip**

"Prototype is about **copying existing instances**, not **creating new ones** via constructors.

🧩 **1. Marker Interface**

✅ **Definition**

A **Marker Interface** is an interface **without any methods or fields** — it **marks a class** to signal something to the JVM or framework.

🧠 **Purpose**

It's used to **convey metadata or behavior** information to the JVM or libraries — like a "tag".

⚙️ **Examples**

- java.io.Serializable
- java.lang.Cloneable
- java.rmi.Remote

These interfaces **don't define methods** but signal special treatment:

- Serializable: tells JVM object can be serialized.
- Cloneable: tells JVM clone() is allowed for that object.

🧩 **Example**

```
class Employee implements Serializable {
   int id;
   String name;
}
```

Here Employee is "marked" as serializable.
If not marked → NotSerializableException during serialization.

💡 **Interviewer Tip:**

Marker interfaces are like “flags” for compiler/runtime.
Annotations (like @FunctionalInterface) replaced most marker interfaces in modern Java.

⚙️ **2. Volatile vs Transient**

Feature	volatile	transient
Purpose	Used in **multi-threading** to ensure visibility of changes across threads.	Used in **serialization** to skip saving a field.
Scope	Concurrency / memory visibility	Serialization
Effect	Forces thread to read from main memory, not CPU cache.	Field won’t be serialized into a file or stream.
Keyword usage	volatile int count;	transient String password;
Example	Thread-safe flag updates.	Hide sensitive data while saving object.

🧩 **Example**

```
class Session {
  volatile boolean active;     // thread visibility
  transient String password;   // skip during serialization
}
```

💡 **Interviewer Tip:**

volatile does not make operations atomic — it only ensures **visibility**, not **atomicity**.
Use AtomicInteger for atomic updates.

🧱 **3. Variable Modifiers in Java**

🔹 Access Modifiers

Modifier	Scope
public	Accessible everywhere
protected	Accessible in same package + subclasses
(default)	Accessible within same package
private	Accessible only within same class

🔹 Non-Access Modifiers

Modifier	Description
static	Belongs to class, not object
final	Value or method/class cannot be changed/overridden
transient	Excluded from serialization
volatile	Thread visibility (multi-threading)
synchronized	Lock-based control for thread safety

💡 Quick Example

```
public class Example {
   public static final double PI = 3.14159;
   private transient String secret;
   static int counter = 0;
}
```

💉 4. Dependency Injection (DI) Types

✅ Definition

Dependency Injection is a design pattern where an object's dependencies are **provided (injected)** rather than created by itself.

Used heavily in **Spring**.

🔹 **Types of Dependency Injection**

Type	Example	Description
Constructor Injection	@Autowired public UserService(UserRepo repo)	Dependencies passed via constructor. Recommended (immutable).
Setter Injection	@Autowired public void setRepo(UserRepo repo)	Dependency set via setter method.
Field Injection	@Autowired private UserRepo repo;	Injected directly into field (less testable).

💡 **Best Practice:**

Prefer **Constructor Injection** → ensures immutability and easier testing.

🗃️ **5. Hibernate – Many-to-Many Mapping**

✅ **Concept**

In **Many-to-Many**, each record in one table can relate to **multiple records** in another — and vice versa.
A **join table** connects both sides.

🧩 **Example**

Let's say:

- One Student can enroll in many Courses
- One Course can have many Students

```
@Entity
public class Student {
  @Id
```

```
    private int id;
    private String name;

    @ManyToMany
    @JoinTable(
        name = "student_course",
        joinColumns = @JoinColumn(name = "student_id"),
        inverseJoinColumns = @JoinColumn(name = "course_id")
    )
    private Set<Course> courses;
}

@Entity
public class Course {
    @Id
    private int id;
    private String title;

    @ManyToMany(mappedBy = "courses")
    private Set<Student> students;
}
```

✅ Hibernate will create a join table: **student_course(student_id, course_id)**

💡 **REST API Integration Example**

```
@GetMapping("/students/{id}/courses")
public List<Course> getCourses(@PathVariable int id) {
    return studentService.getCoursesForStudent(id);
}
```

🧮 6. SQL Query – Find Employee with 7th Highest Salary

✅ Approach 1: Using LIMIT (MySQL)

```
SELECT DISTINCT salary
FROM employee
ORDER BY salary DESC
LIMIT 6, 1;
```

- Skips 6 salaries → picks 7th one.

✅ Approach 2: Using Subquery (Generic SQL)

```
SELECT MIN(salary)
FROM (
 SELECT DISTINCT salary
 FROM employee
 ORDER BY salary DESC
 FETCH FIRST 7 ROWS ONLY
) AS temp;
```

✅ Approach 3: Using Window Function (Oracle/Postgres)

```
SELECT salary
FROM (
 SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
 FROM employee
) ranked
WHERE rnk = 7;
```

🧩 7. Java Object Reference Checks

✅ **Example**

A a = new A();

A b = a;

A c = new A();

Comparison	Result	Explanation
a == b	✅ true	Both refer to same object in memory.
a == c	❌ false	Different objects, even if values same.
a.equals(b)	true (if equals() not overridden)	Default checks reference.
a.equals(c)	false	Unless overridden to compare field values.

💡 **Interviewer Tip:**

- == → compares **references**
- .equals() → compares **values** (if overridden)

🧠 **8. Built-in Functional Interfaces (Top 5)**

Java 8 provides many in java.util.function package.

Interface	Method	Purpose	Example
Function<T, R>	R apply(T t)	Takes input, returns output	Function<String, Integer> len = s -> s.length();
Predicate<T>	boolean test(T t)	Used for conditional checks	Predicate<Integer> even = n -> n % 2 == 0;
Consumer<T>	void accept(T t)	Consumes value, no return	Consumer<String> print = s -> System.out.println(s);
Supplier<T>	T get()	Supplies value, no input	Supplier<Double> rand = () -> Math.random();

Interface	Method	Purpose	Example
BiFunction<T, U, R>	R apply(T t, U u)	Takes 2 inputs, returns output	BiFunction<Integer, Integer, Integer> sum = (a,b)->a+b;

💡 **Bonus: UnaryOperator and BinaryOperator are variants of Function.**

⚡ **9. Why We Use Lambda Expressions & Functional Interfaces**

✅ **Functional Interface**

- Interface with **exactly one abstract method** (SAM – Single Abstract Method).
- Enables **lambda expressions**.
- Annotated with @FunctionalInterface.

Example:

```
@FunctionalInterface
interface Greeting {
   void sayHello(String name);
}
```

✅ **Lambda Expression**

A short way to write anonymous methods (functional style).

```
Greeting g = name -> System.out.println("Hello " + name);
g.sayHello("Suraj");
```

💡 **Why we use them**

Reason	Description
🧠 **Less boilerplate**	No need for anonymous inner classes
⚡ **Functional style**	More readable stream-based code

Reason	Description
🧩 **Integration with Streams API**	Enables map, filter, reduce, etc.
🔧 **Cleaner callbacks**	Used in async operations, listeners

✅ **Example (Stream + Lambda)**

```
List<Integer> list = Arrays.asList(1,2,3,4,5);

list.stream()
  .filter(n -> n % 2 == 0)
  .map(n -> n * n)
  .forEach(System.out::println);
```

🧩 **1. java.lang.Object – Core Methods**

Every Java class **implicitly extends Object**, which defines **basic behavior common to all objects**.

Method	Purpose	Example
toString()	Returns string representation of the object.	"Employee[id=1,name=John]"
equals(Object obj)	Compares two objects for equality.	emp1.equals(emp2)
hashCode()	Returns integer hash (used in HashMap, Set).	Ensures hashCode() consistent with equals()
getClass()	Returns runtime Class object.	emp.getClass().getName()
clone()	Creates a copy of the object (if Cloneable).	obj.clone()
finalize()	Called by GC before object destruction (deprecated).	Cleanup (rarely used).
wait(), notify(), notifyAll()	Used for thread synchronization.	Wait/notify communication between threads.

🧩 **Example**

```
class Employee {
   int id;
   String name;

   @Override
   public String toString() {
      return id + " - " + name;
   }

   @Override
   public boolean equals(Object o) {
      if (this == o) return true;
      if (!(o instanceof Employee e)) return false;
      return id == e.id && name.equals(e.name);
   }

   @Override
   public int hashCode() {
      return Objects.hash(id, name);
   }
}
```

✅ Always override equals() and hashCode() together.

⚙️ **2. Java Varargs (...)**

✅ **Definition**

Varargs (Variable Arguments) let you **pass variable number of arguments** to a method.

🧩 Syntax

```
void printNumbers(int... nums) {
   for (int n : nums)
      System.out.println(n);
}

printNumbers(1, 2, 3, 4);
```

- Internally, compiler converts varargs into **array** (int[] nums).
- You can still pass an array directly.

⚠️ Rules

- Only **one vararg per method**.
- It must be **the last parameter** in the method signature.

🌎 3. Global Meeting Scheduling (Time Zone Management)

When building a **meeting application** (like Google Meet or Zoom), you need **global time zone consistency**.

✅ Problem

If a user in **India** schedules a meeting at **10 AM IST**, users in **US** or **UK** should see the correct **local time** automatically.

✅ Solution Strategy

1. **Always store time in UTC** in your database.
   → e.g., 2025-11-06T04:30:00Z
2. **Convert time zones** only at **display time** using user's locale.

🧩 Example (Java 8+ Time API)

```
ZonedDateTime indiaTime = ZonedDateTime.of(
    2025, 11, 6, 10, 0, 0, 0, ZoneId.of("Asia/Kolkata")
);

ZonedDateTime utc = indiaTime.withZoneSameInstant(ZoneId.of("UTC"));
ZonedDateTime usTime = indiaTime.withZoneSameInstant(ZoneId.of("America/New_York"));

System.out.println("UTC: " + utc);
System.out.println("US Time: " + usTime);
```

✅ Output will show same moment in different time zones.
Store utc in DB and convert as needed in UI.

💡 **Best Practice**

- Always use ZonedDateTime or OffsetDateTime
- Never store local time — always store UTC.

🧱 **4. Java 25 – Key Features (as of 2025)**

Java 25 (LTS expected) continues improving **performance, pattern matching, and memory management**.

Feature	Description
Value Objects (Valhalla)	Lightweight, identity-less objects for performance.
Universal Generics	Allow generics for primitives (e.g., List<int>).
Enhanced Pattern Matching	Simplifies switch expressions and instanceof.
String Templates (Preview)	Inline variable embedding like Python f-strings.

Feature	Description
Virtual Threads (Loom)	High-scale concurrency with lightweight threads.
Scoped Values	Replace ThreadLocal in virtual threads safely.
Foreign Function & Memory API (Panama)	Call native libraries without JNI.
Garbage Collection Improvements	ZGC and G1 enhancements for latency reduction.

🧩 Example – String Templates

String name = "Suraj";

String msg = STR."Hello, \{name}! Welcome to Java 25.";

System.out.println(msg);

Output → Hello, Suraj! Welcome to Java 25.

📊 5. Reporting Features in Java

✅ Common Libraries

Tool	Description	Use Case
JasperReports	Most popular open-source reporting engine.	PDF, Excel, HTML, etc.
Apache POI	Generate Excel/Word reports.	Spreadsheet reports.
iText / OpenPDF	Generate and manipulate PDFs.	PDF invoices, receipts.

🧩 Jasper Example Flow

1. Design .jrxml template in *Jasper Studio*
2. Compile to .jasper file
3. Fill with data → JasperFillManager
4. Export as PDF/HTML

JasperPrint jp = JasperFillManager.fillReport("report.jasper", params, connection);

JasperExportManager.exportReportToPdfFile(jp, "report.pdf");

🖨️ **Printing Management**

Use **java.awt.print** package:

PrinterJob job = PrinterJob.getPrinterJob();

job.setPrintable(myPrintable);

if (job.printDialog()) {

  job.print();

}

🌐 **6. Design Simple API using Servlet (Return JSON)**

✅ **Steps**

1. Create a **Servlet**
2. Set **response type to JSON**
3. Write JSON using PrintWriter

🧩 **Example**

@WebServlet("/user")

public class UserServlet extends HttpServlet {

  protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws IOException {

    resp.setContentType("application/json");

    PrintWriter out = resp.getWriter();

    String json = """

    {

      "id": 1,

```
        "name": "Suraj",
        "role": "Developer"
    }
    """;

    out.print(json);
    out.flush();
  }
}
```

✅ Output →

{"id":1,"name":"Suraj","role":"Developer"}

In production, use libraries like **Jackson** or **Gson** for JSON serialization.

🗺️ 7. Map.merge() Function

Introduced in **Java 8**, simplifies updating values in a map.

✅ Syntax

V merge(K key, V value, BiFunction<V, V, V> remappingFunction)

- If key absent → inserts value.
- If key present → combines old + new value.

🧩 Example

```
Map<String, Integer> scores = new HashMap<>();
scores.put("John", 50);

scores.merge("John", 10, Integer::sum); // adds 10
scores.merge("Mary", 40, Integer::sum); // inserts new
```

System.out.println(scores); // {John=60, Mary=40}

✅ Great for word count, aggregations, or frequency maps.

🔍 8. Reflection in Java

✅ Definition

Reflection allows inspection and modification of classes, fields, methods, and constructors at **runtime**.

Located in java.lang.reflect package.

🧩 Example – Get All Info About a Class

```
import java.lang.reflect.*;

class Person {
   private String name;
   public int age;

   public Person(String name, int age) {}
   public void show() { System.out.println("Hello"); }
}

public class ReflectionDemo {
   public static void main(String[] args) throws Exception {
      Class<?> cls = Class.forName("Person");

      System.out.println("Class Name: " + cls.getName());

      System.out.println("\nFields:");
      for (Field f : cls.getDeclaredFields())
```

```
        System.out.println("  " + f.getName() + " : " + f.getType());

    System.out.println("\nConstructors:");
    for (Constructor<?> c : cls.getDeclaredConstructors())
        System.out.println("  " + c);

    System.out.println("\nMethods:");
    for (Method m : cls.getDeclaredMethods())
        System.out.println("  " + m.getName());
  }
}
```

✅ Output:

```
Class Name: Person
Fields:
 name : class java.lang.String
 age : int
Constructors:
 public Person(java.lang.String,int)
Methods:
 show
```

⚠️ **Use Cases**

- Frameworks like **Spring**, **Hibernate** use reflection for dependency injection and ORM mapping.
- **Caution:** slower, less type-safe, and bypasses encapsulation.

🧩 **1. Finite vs Infinite Collection**

✅ **Finite Collection**

A collection with a **fixed, countable number of elements**.

📘 **Examples:**

- List.of(1, 2, 3, 4, 5)
- Array of employees loaded from DB

💡 Once you reach the end → no more elements.

✅ **Infinite Collection**

A collection (often **Stream**) that can **generate endless data**.

📘 **Examples:**

Stream<Integer> infinite = Stream.iterate(0, n -> n + 1);

💡 Used for **data generation**, simulation, or reactive systems.
You must use **short-circuiting** operations (limit(), findFirst(), etc.)

⚠️ **Note:**

Collections in Java (List, Set, etc.) are **finite**,
but **Streams** can be **finite or infinite**.

⚙️ **2. return in try/catch/finally — Execution Order**

✅ **Rule**

- finally block **always executes** — even if there's a return in try or catch.
- Only exceptions:
    - When JVM **exits** (System.exit(0)).
    - When a **fatal error** occurs (e.g., power loss, StackOverflowError).

🧩 **Example**

```
public int test() {
  try {
    return 10;
```

```
    } catch (Exception e) {
        return 20;
    } finally {
        System.out.println("Finally executed");
    }
}
```

✅ Output:

Finally executed

Returned: 10

⚠️ **What if finally also has return?**

```
try {
    return 10;
} finally {
    return 30;
}
```

✅ finally return **overrides** everything → returns **30**.

💡 **Best Practice:**
Never return from finally. It confuses logic and hides exceptions.

🔁 **3. map() vs flatMap()**

Both are **Stream transformation methods** — but with different outputs.

Method	Input	Output	Used for
map()	Function → one-to-one	Stream of transformed elements	Transform values
flatMap()	Function → one-to-many (Stream)	Flattened single Stream	Flatten nested structures

---

🧩 **Example**

```
List<List<Integer>> nums = List.of(
   List.of(1, 2),
   List.of(3, 4)
);

List<Integer> mapped = nums.stream()
   .map(List::stream)
   .count(); // Stream<Stream<Integer>>

List<Integer> flatMapped = nums.stream()
   .flatMap(List::stream)
   .toList(); // Stream<Integer>
```

✅ flatMap flattens nested collections.
💡 Think of it as “merge all inner streams into one”.

🧱 **4. String Block (Text Block)**

Introduced in **Java 15** to simplify **multi-line strings**.

✅ **Syntax**

```
String json = """
{
 "name": "Suraj",
 "role": "Developer"
}
""";
```

✅ Easier to read and avoids \n or + concatenations.

💡 **Benefits**

- Clean multi-line formatting
- Supports indentation & escaping
- Internally stored as **String**

🧩 5. class vs Class

Term	Meaning
class	Keyword — used to define a class.
Class	Java class (java.lang.Class) — represents **runtime type information** of any object.

🧩 Example

```
Employee e = new Employee();

Class<?> c = e.getClass(); // Class object

System.out.println(c.getName()); // prints package.Employee
```

💡 Every loaded class in JVM has one Class object (metadata).

⚙️ 6. final Keyword with Different Variables

Type	Meaning	Example
Local Variable	Can be assigned once	final int x = 10;
Instance Variable	Must be assigned in constructor or inline	final String name;
Static Variable	Assigned once in static block or inline	static final int MAX = 100;

🧩 Example

```
class Test {
   final int a;          // instance final
   static final int b;     // static final
   final int x = 5;       // inline initialization
```

```
  static {
    b = 10;          // static block initialization
  }

  Test() {
    a = 20;          // constructor initialization
  }
}
```

✅ You can’t change a final variable after assignment.
💡 Common for constants (public static final).

🧠 7. Use of String[] args in main()

✅ Definition

Command-line arguments passed to Java program at runtime.

```
public static void main(String[] args) {
  for (String arg : args)
    System.out.println(arg);
}
```

Run:

```
java MyProgram Suraj 25
```

Output:

```
Suraj
25
```

💡 Frameworks like **Spring Boot** use these args for runtime configuration.

🌐 8. Spring App Starts but All Endpoints Return 404

✅ Common Causes & Solutions

Cause	Description	Fix
Wrong Package Structure	@SpringBootApplication in a package that doesn't scan controllers	Move Application.java to root package.
Missing @RestController / @Controller	Bean not registered	Add annotation to class.
Wrong Request Mapping	URL mismatch	Verify @RequestMapping path.
Context Path Issue	Custom server.servlet.context-path set	Check in application.properties.
Port Mismatch	API running on different port	Verify logs (Tomcat started on port...).
Classpath Issue	Wrong jar or dependency	Ensure spring-boot-starter-web added.
Returning JSP/HTML	No spring-boot-starter-thymeleaf	Add required template dependency.

🧩 **Pro Debug Steps**

1. Check startup logs → look for "Mapped" URLs.
2. Access http://localhost:8080/actuator/mappings (if actuator enabled).
3. Verify your @ComponentScan scope.

🕒 **9. Initial Delay – Controller Startup**

If controller is **slow to initialize**, possible reasons:

Cause	Solution
Heavy initialization logic in constructor	Move logic to @PostConstruct or background thread.
Lazy initialization	Use spring.main.lazy-initialization=false.

Cause	Solution
Database connection slow	Test DB connectivity.
Large dependency injection tree	Optimize bean structure.

✅ You can measure startup time using Spring Boot Actuator metrics or ApplicationListener<ApplicationReadyEvent>.

🧱 **10. SQL – Finding Second / Nth Highest Salary**

✅ **Method 1: Using Subquery**

```
SELECT MAX(salary)
FROM employee
WHERE salary < (SELECT MAX(salary) FROM employee);
```

👉 **Second highest**

✅ **Method 2: Nth Highest (Generic)**

```
SELECT DISTINCT salary
FROM employee e1
WHERE N - 1 = (
  SELECT COUNT(DISTINCT salary)
  FROM employee e2
  WHERE e2.salary > e1.salary
);
```

Replace N → e.g. 7 for 7th highest.

🧱 **2. Grouping & Aggregation in Streams**

The **Collectors** class provides grouping and aggregation functions.

✅ 1️⃣ **Group by Single Field**

```
Map<String, List<Employee>> byDept = employees.stream()
   .collect(Collectors.groupingBy(Employee::getDepartment));
```

→ {HR=[...], IT=[...], SALES=[...]}

✅ 2️⃣ Group and Count

```
Map<String, Long> countByDept = employees.stream()
   .collect(Collectors.groupingBy(Employee::getDepartment, Collectors.counting()));
```

→ {HR=2, IT=3, SALES=1}

✅ 3️⃣ Group and Aggregate (Sum)

```
Map<String, Integer> totalSalary = employees.stream()
   .collect(Collectors.groupingBy(Employee::getDepartment,
         Collectors.summingInt(Employee::getSalary)));
```

→ {HR=50000, IT=90000}

✅ 4️⃣ Multi-level Grouping

```
Map<String, Map<String, List<Employee>>> grouped =
   employees.stream()
      .collect(Collectors.groupingBy(Employee::getDepartment,
            Collectors.groupingBy(Employee::getRole)));
```

→ Nested grouping.

✅ 5️⃣ Aggregation Without Grouping

```
int total = employees.stream()
   .mapToInt(Employee::getSalary)
   .sum();
```

💡 **Tip**

Use reducing() and summarizingInt() for complex custom aggregations.

🧪 **3. High-Level Unit Testing with JUnit (Annotations Overview)**

JUnit 5 = Modern Java testing framework.

🧩 **Common Annotations**

Annotation	Purpose
@Test	Marks method as test case
@BeforeEach	Runs before every test
@AfterEach	Runs after every test
@BeforeAll	Runs once before all tests (static)
@AfterAll	Runs once after all tests
@DisplayName("...")	Custom name for test
@Disabled	Temporarily skip test
@ParameterizedTest	Run same test with different inputs
@Tag("integration")	Group tests by category

🧩 **Example**

```
class CalculatorTest {
  Calculator calc;

  @BeforeEach
  void init() { calc = new Calculator(); }

  @Test
```

```
    void testAdd() {
        assertEquals(10, calc.add(7, 3));
    }

    @AfterEach
    void cleanup() {
        calc = null;
    }
}
```

🧠 4. Mocking Data with Mockito

Mockito = Mock framework for simulating dependencies.

✅ Common Annotations

Annotation	Meaning
@Mock	Creates a mock object
@InjectMocks	Injects mocks into tested class
@Spy	Partial mock (real + mocked)
@Captor	Captures arguments passed to mocks

🧩 Example

```
@ExtendWith(MockitoExtension.class)
class UserServiceTest {

    @Mock
    UserRepository repo;
```

```
    @InjectMocks
    UserService service;

    @Test
    void testFindUser() {
        when(repo.findById(1)).thenReturn(Optional.of(new User(1, "Suraj")));

        User u = service.getUser(1);

        assertEquals("Suraj", u.getName());
        verify(repo).findById(1);
    }
}
```

💡 **Mockito verifies interactions** (verify()), not just data.

🌐 5. Integration Testing (High Level)

Integration tests validate **full flow** — from controller → service → repository.

✅ Spring Boot Example

```
@SpringBootTest(webEnvironment = WebEnvironment.RANDOM_PORT)
@AutoConfigureMockMvc
class UserControllerIT {

    @Autowired
    private MockMvc mockMvc;

    @Test
    void testGetUsers() throws Exception {
```

```
    mockMvc.perform(get("/users"))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$[0].name").value("Suraj"));
  }
}
```

✅ Runs full Spring context.
✅ Verifies endpoint + data together.
✅ Use @Sql to preload DB data before tests.

⚙️ 6. Spring Boot Property Resolution Order

Spring Boot resolves configuration from **multiple sources** — in a specific **precedence order**.

🧩 Order (High → Low Priority)

Priority	Source
1️⃣	**Command-line arguments**
2️⃣	**Environment variables**
3️⃣	**application.properties / .yml** (inside src/main/resources)
4️⃣	**Profile-specific files** (application-dev.yml)
5️⃣	**External config files** (/config folder, jar dir)
6️⃣	**@PropertySource annotations**
7️⃣	**Default values in @Value or code**

💡 Example

```
@Value("${server.port:8080}")
private int port;
```

If no property found → default 8080 used.

🧱 7. DTO Versioning in Spring Boot

When APIs evolve, DTOs (Data Transfer Objects) must handle **backward compatibility**.

✅ Approaches

Approach	Example	Best for
URL Versioning	/api/v1/users, /api/v2/users	Simpler REST pattern
Header-based	Accept: application/vnd.company.app-v2+json	Advanced version negotiation
Parameter-based	/users?version=2	Easy backward compatibility
DTO Class Versioning	UserDTOv1, UserDTOv2	When field structure changes

---

🧩 Example (URL-based)

```
@RestController
@RequestMapping("/api/v1/users")
public class UserControllerV1 {
   @GetMapping public List<UserDTOv1> getAll() { ... }
}

@RestController
@RequestMapping("/api/v2/users")
public class UserControllerV2 {
   @GetMapping public List<UserDTOv2> getAll() { ... }
}
```

✅ Keeps API backward-compatible
✅ You can maintain different DTO versions cleanly

🧩 8. Print Message Without main() Method

✅ 1️⃣ Using Static Block

```
class Hello {
  static {
    System.out.println("Hello, Java!");
    System.exit(0); // prevents NoSuchMethodError
  }
}
```

✅ Output:

```
Hello, Java!
```

✅ 2️⃣ Using Static Initializer in Enum/Class

```
class Demo {
  static String message = print();
  static String print() {
    System.out.println("Printed without main");
    System.exit(0);
    return "";
  }
}
```

💡 JVM executes static blocks before main() —
hence can print even if main() is missing.

🧩 1. Top 3 JSON Parsers in Spring Boot

Spring Boot uses **Jackson** by default for JSON serialization/deserialization, but there are others too.

JSON Parser	Description	Use Case / Pros
🦊 **Jackson** (default)	Powerful, flexible, supports annotations	Default in Spring Boot (com.fasterxml.jackson)
Gson (by Google)	Lightweight, simple API, used in Android	Good for small apps or legacy systems
🦋 **Json-B / Johnzon / Moshi**	JSON Binding (Jakarta EE), Moshi for Android	Used when following JSR-367 standard or in microservices with Quarkus

---

✅ **Example – Jackson**

```
@RestController
public class UserController {
  @GetMapping("/user")
  public User getUser() {
    return new User(1, "Suraj");
  }
}
```

Spring Boot automatically converts User → JSON using **Jackson**.

Custom config:

```
spring.jackson.property-naming-strategy: SNAKE_CASE
spring.jackson.date-format: yyyy-MM-dd
```

💡 **Tip**

Jackson = Spring default
Gson = Simple/lightweight alternative
Json-B = Standard in Jakarta EE

⚙️ **2. Blank Final in Java**

✅ **Definition**

A **blank final variable** is a final variable **declared but not initialized** at declaration time — it **must be initialized later**, usually in a **constructor**.

🧩 Example

```
class Employee {
   final int empId; // blank final

   Employee(int id) {
      empId = id; // initialized in constructor
   }
}
```

✅ Legal because empId gets initialized before object creation finishes.

⚠️ If you don't initialize a blank final → Compile-time Error

```
class Test {
   final int x;  // ERROR if never assigned in constructor
}
```

💡 Why use blank final?

- For **immutability** while allowing **constructor-based initialization**
- Common in **dependency injection** and **thread-safe objects**

📄 3. Pagination in JPA – 2 Best Practices

Pagination prevents loading millions of records at once and improves DB & app performance.

✅ 1️⃣ Use Pageable and Page Interface

Spring Data JPA supports pagination directly.

```
public interface EmployeeRepository extends JpaRepository<Employee, Long> {
   Page<Employee> findByDepartment(String dept, Pageable pageable);
}
```

Controller:

```
@GetMapping("/employees")
public Page<Employee> getEmployees(
      @RequestParam int page,
      @RequestParam int size) {
   return repo.findAll(PageRequest.of(page, size, Sort.by("id").ascending()));
}
```

✅ Returns a Page object with:

- getContent() → list of items
- getTotalElements()
- getTotalPages()

✅ 2️⃣ **Best Practices**

Practice	**Explanation**
a. Use Index-based Sort Columns	Always sort by indexed column (e.g., id) for fast paging
b. Avoid deep pagination (large offsets)	Use **keyset pagination** or "Seek method" for better performance:
WHERE id > last_seen_id ORDER BY id LIMIT n	
c. Use Projections	Fetch only required columns using DTO or projection
d. Return Metadata	Return total count, page number, etc. in response
e. Cache counts	Counting total records can be expensive — cache if possible

---

💡 **Example JSON Response**

```
{
 "content": [{ "id": 1, "name": "Suraj" }],
 "pageable": { "pageNumber": 0, "pageSize": 10 },
 "totalPages": 20,
 "totalElements": 200
}
```

🪵 **4. SLF4J vs Log4j**

Feature	SLF4J	Log4j / Log4j2
Type	Logging **Facade (API)**	**Implementation**
Purpose	Provides abstraction (unified logging API)	Actual logger that writes logs
Used By	Frameworks like Spring, Hibernate	Can be used standalone
Syntax	log.info("Message")	logger.info("Message")
Config File	Uses implementation's config (e.g., Logback XML)	log4j2.xml, log4j.properties
Thread Safety	Yes	Yes
Performance	Depends on backend	Log4j2 is async & faster

✅ **In Spring Boot**

- Uses **SLF4J + Logback** by default
- You can switch to **Log4j2** easily via dependency

```
<dependency>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-log4j2</artifactId>
```

</dependency>

💡 **Rule of Thumb:**

SLF4J = Interface → “log API”
Log4j2 / Logback = Implementation → “engine writing logs”

🔗 **5. Serialization Interface in Java**

✅ **Definition**

java.io.Serializable is a **marker interface** (no methods) that allows converting an object → **byte stream** for:

- File storage
- Network transmission
- Caching

🧩 **Example**

```
class Employee implements Serializable {
   private static final long serialVersionUID = 1L;
   int id;
   String name;
}
```

Serialization

```
ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream("emp.ser"));
out.writeObject(new Employee(1, "Suraj"));
```

Deserialization

```
ObjectInputStream in = new ObjectInputStream(new FileInputStream("emp.ser"));
Employee e = (Employee) in.readObject();
```

💡 **Best Practices**

Practice	Why
Add serialVersionUID	Avoid version mismatch errors
Avoid serializing sensitive data	Use transient keyword
Prefer JSON serialization (Jackson) for APIs	Portable & human-readable

---

⏰ 6. Handling Scheduler Failure in Spring Boot

Schedulers (e.g., @Scheduled) can fail due to DB issues, network timeouts, or application restarts.
Here's how to make them **resilient and self-healing** 👇

✅ Common Causes

Cause	Example
DB/API down	Scheduler throws exception
Task overlap	Job starts before previous one ends
App restart	Missed scheduled executions
Memory/Thread exhaustion	Long-running tasks block scheduler pool

---

✅ Best Practices to Handle Scheduler Failures

🧩 1️⃣ Use @Scheduled with Error Handling

```
@Scheduled(fixedRate = 60000)
public void fetchData() {
  try {
    service.callExternalApi();
  } catch (Exception e) {
    log.error("Scheduler failed: {}", e.getMessage(), e);
  }
}
```

✅ Prevents scheduler thread crash.

🧩 2️⃣ Use @EnableScheduling + Thread Pool

```
@Configuration
@EnableScheduling
public class SchedulerConfig implements SchedulingConfigurer {
  @Override
  public void configureTasks(ScheduledTaskRegistrar registrar) {
    ThreadPoolTaskScheduler scheduler = new ThreadPoolTaskScheduler();
    scheduler.setPoolSize(5);
    scheduler.initialize();
    registrar.setTaskScheduler(scheduler);
  }
}
```

✅ Prevents blocking if one task hangs.

🧩 3️⃣ Add Retry Mechanism

Integrate with **Spring Retry** or **Resilience4j**.

```
@Retryable(
  value = Exception.class,
  maxAttempts = 3,
  backoff = @Backoff(delay = 2000)
)
public void callApi() { ... }

@Recover
public void recover(Exception e) {
  log.error("Job failed after retries: {}", e.getMessage());
```

}

🧩 4️⃣ Use Job Monitoring

- Expose metrics via **Actuator**
- Send alerts/slack messages when job fails
- Example: push to Prometheus or email on error.

🧩 5️⃣ Use Persistent Job Scheduler (for critical tasks)

For production systems, prefer:

- **Quartz Scheduler**
- **Spring Batch**
- **Resilient Queues (Kafka / RabbitMQ)**

✅ They persist job state and retry automatically after restarts.

💡 Tip

For mission-critical scheduled jobs — never rely solely on @Scheduled.
Use **Quartz + database-backed triggers** for guaranteed execution.

Excellent 👏 — you're now covering **advanced Spring Boot performance, concurrency, and system diagnostics topics** — exactly the kind that appear in **senior Java developer interviews or SRE/architect discussions**.

Let's go step-by-step with **clear explanations + real-world examples + best practices** 👇

🚀 1. Identifying Performance Issues in Spring Boot

When your Spring Boot app slows down or consumes high CPU/memory, the key is to diagnose **where the bottleneck lies** —
CPU, memory, threads, DB, logs, or distributed tracing.

✅ Main Areas to Investigate

Area	Symptoms	Tools / Techniques
Memory (Heap/GC)	OutOfMemoryError, GC pause	JProfiler, VisualVM, Actuator /heapdump
CPU	High CPU usage, slow requests	JProfiler, jcmd, top, async profiling
Threads	Deadlock, stuck threads	Thread dump (jstack), JProfiler
Database	Slow queries, timeouts	Actuator metrics, spring.datasource.hikari.*, APM
Logs	Missing/too many logs	Logback async, correlation IDs
Tracing (Latency)	Slow external API or microservice	Spring Cloud Sleuth + Zipkin
GC/Heap	Frequent full GC	-XX:+PrintGCDetails, JVisualVM

⚙️ Actuator Endpoints (built-in profiling)

Enable Actuator in application.yml:

management.endpoints.web.exposure.include: "*"

Useful endpoints:

- /actuator/metrics → memory, thread, GC, CPU metrics
- /actuator/health → component health
- /actuator/threaddump → check deadlocks
- /actuator/heapdump → analyze memory leak offline

🧠 Using JProfiler (for local profiling)

1. Attach JProfiler to your Spring Boot process.
2. Analyze:
    - CPU hotspots → which methods consume time
    - Heap allocations → detect leaks

- JDBC calls → slow queries
- Threads → blocked/waiting states

💡 **Key Insight:**

If CPU is high but DB is idle → app-level issue.
If DB latency is high → bottleneck in query/indexing.

⚡ **Distributed Tracing: Zipkin + Sleuth**

For microservices, use **Spring Cloud Sleuth** for request tracking.

<dependency>

<groupId>org.springframework.cloud</groupId>

<artifactId>spring-cloud-starter-zipkin</artifactId>

</dependency>

Then, every request carries:

- **Trace ID** (unique for request)
- **Span ID** (per microservice call)

In Zipkin UI, visualize time spent per service:
🕒 API → Service A → Service B → Database

✅ Helps detect *which service is slow* in the chain.

🔍 **Summary: Key Profiling Flow**

1. **Start with Actuator metrics**
2. **Check thread & heap dumps**
3. **Use JProfiler locally**
4. **Add Sleuth + Zipkin for distributed latency**
5. **Optimize DB queries / indexes**
6. **Review logs (async, structure)**

📝 **2. Virtual Threads in Spring Boot (Project Loom)**

✅ What are Virtual Threads

- Introduced in **Java 21** (java.lang.Thread.ofVirtual()).
- They are **lightweight threads** managed by the JVM, not OS.
- Allow thousands of concurrent tasks without blocking OS threads.

🧩 Example

```
var executor = Executors.newVirtualThreadPerTaskExecutor();

executor.submit(() -> {

  System.out.println("Running on " + Thread.currentThread());

});

executor.close();
```

✅ Each task gets its own virtual thread → minimal memory, no blocking.

💡 In Spring Boot

- Use **Spring Boot 3.2+** (supports virtual threads).
- Add property:

```
spring.threads.virtual.enabled: true
```

✅ Benefits

Benefit	Description
Lightweight	1 OS thread can handle 1000s of virtual threads
Great for I/O-heavy apps	WebFlux, database, HTTP calls
Easier than reactive code	Keeps imperative style with concurrency
Minimal memory footprint	Each thread ≈ 2KB stack

⚠️ Limitations

- Not beneficial for **CPU-bound tasks**.

- Must use libraries that release I/O threads properly (no native blocking).

👥 3. Concurrent Sessions in Spring Boot

Spring Security allows multiple sessions per user — but often you want to **limit or track them**.

✅ Enable Session Management

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {
  @Override
  protected void configure(HttpSecurity http) throws Exception {
    http.sessionManagement()
      .maximumSessions(1)
      .maxSessionsPreventsLogin(true);
  }
}
```

- maximumSessions(1) → only 1 session allowed
- maxSessionsPreventsLogin(true) → rejects new login if one exists

---

💡 Limitations

Limitation	Description
Works only if session stored centrally	e.g., Redis-backed Spring Session
Doesn't persist across service restarts	Sessions are in-memory unless external store used
Needs sticky sessions or shared storage	Otherwise, multi-instance logouts fail

---

✅ **Fix for Multi-instance Deployment**

Use **Spring Session + Redis**:

<dependency>

<groupId>org.springframework.session</groupId>

<artifactId>spring-session-data-redis</artifactId>

</dependency>

✅ Enables **shared session store** → consistent session limits cluster-wide.

📜 **4. @EventListener vs @TransactionalEventListener**

Feature	**@EventListener**	**@TransactionalEventListener**
When triggered	Immediately when event is published	After transaction commit
Typical use case	Notifications, async events	DB-based updates, audit logs
Example event	UserCreatedEvent → send email	OrderPlacedEvent → update inventory after commit

---

🧩 **Example**

@Component

public class MyListener {

  @EventListener

  public void handleNormal(Event e) {

    System.out.println("Triggered immediately");

  }

  @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)

  public void handleTransactional(Event e) {

```
        System.out.println("Triggered after DB commit");
    }
}
```

✅ Use @TransactionalEventListener to ensure event runs **only if transaction succeeds**.

🔁 5. Circular Dependency in Spring Boot

⚠️ Problem

Two beans depend on each other → Spring can't decide initialization order.

```
@Component
class A {
    @Autowired
    B b;
}

@Component
class B {
    @Autowired
    A a;
}
```

Error:

BeanCurrentlyInCreationException: Requested bean is currently in creation

---

✅ Solutions

Approach	Example
1. Use @Lazy	@Autowired @Lazy private B b;
2. Use constructor injection and redesign	Break cyclic reference

Approach	Example
3. Use interfaces or events	Decouple communication
4. Use setter injection	One side lazy-loaded

🧩 Better Design Example

Instead of A ↔ B direct:

@Component class A { private final CommonService s; }

@Component class B { private final CommonService s; }

✅ Both depend on a shared service — no circular chain.

🧠 6. Incorrect Bean Scope Issues

Scope	Description	Common Pitfall
singleton	One instance per app	Holding request data in singleton bean
prototype	New instance per injection	Misused → memory leak
request	One per HTTP request	Not available in async/background threads
session	One per user session	Not valid in non-web contexts

⚠️ Example Problem

@Component

@Scope("singleton")

public class UserContext {

  private String username; // ❌ shared across users

}

✅ Fix:

@Component

@Scope("request")

```
public class UserContext {
   private String username;
}
```

🔁 7. Labeled and Unlabeled break Statements

✅ Unlabeled break

Breaks the **nearest loop**.

```
for (...) {
   if (x == 5) break; // exits only inner loop
}
```

✅ Labeled break

Breaks **outer loop**.

```
outer:
for (int i = 0; i < 3; i++) {
   for (int j = 0; j < 3; j++) {
      if (j == 2) break outer; // exits both loops
   }
}
```

💡 Useful for nested loops where early exit is needed.

📩 8. Kafka Message Filtering

Kafka filtering prevents unnecessary messages from reaching consumers.

✅ Approaches

Method	Description
1. Consumer-side Filtering	Read all messages, process only needed ones
2. Using RecordFilterStrategy	Filter before listener handles

Method	Description
3. Using Topic-level separation	Route different types of data to different topics

---

🧩 Example – Spring Kafka Filter

```
@Bean
public ConcurrentKafkaListenerContainerFactory<String, String>
kafkaListenerContainerFactory() {
   ConcurrentKafkaListenerContainerFactory<String, String> factory = new
ConcurrentKafkaListenerContainerFactory<>();
   factory.setConsumerFactory(consumerFactory());
   factory.setRecordFilterStrategy(record -> !record.value().contains("VALID"));
   return factory;
}
```

✅ Messages without "VALID" are skipped (not acknowledged).
💡 Filtered messages can also be sent to a **dead-letter topic** if needed.

Perfect — you're now in **"Advanced Spring Boot + Kafka + Concurrency + Security + Lifecycle"** territory — the level that impresses interviewers 👏

Below is your **complete and concise interview-prep explanation** for each topic — including **examples, internal flow, and best practices** 👇

📨 1. Exactly Once Delivery Semantics in Kafka

✅ Concept

Kafka aims for **"exactly-once" semantics (EOS)** — ensuring a message is **processed once and only once**, even in case of retries or failures.

Normally Kafka provides:

- **At most once** → message lost on failure
- **At least once** → message reprocessed (duplicate)
- **Exactly once** → no loss, no duplicates ✅

⚙️ How It Works

Achieved through **Idempotent Producer + Transactional Writes**

Component	Role
Idempotent Producer	Ensures same message not written twice
Transactional Producer	Groups multiple send + consume operations as one atomic transaction
Consumer Offset in TX	Offsets are committed atomically with the transaction

---

🧩 Example

```
Properties props = new Properties();
props.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true);
props.put(ProducerConfig.TRANSACTIONAL_ID_CONFIG, "tx-123");

KafkaProducer<String, String> producer = new KafkaProducer<>(props);
producer.initTransactions();

producer.beginTransaction();
try {
   producer.send(new ProducerRecord<>("orders", "key", "value"));
   producer.commitTransaction();
} catch (Exception e) {
   producer.abortTransaction();
}
```

✅ Guarantees:
Message written once, even after retries.

💡 Spring Kafka

Use:

spring.kafka.producer.transaction-id-prefix: tx-

spring.kafka.producer.properties.enable.idempotence: true

⚠️ Limitations

- Slight performance cost (2-phase commit)
- Broker config must support transaction.state.log.replication.factor ≥ 3
- Needs careful consumer setup

🔁 2. Transaction Management in Kafka

Kafka transactions ensure **atomicity** between producing and consuming messages — "consume-process-produce" loop.

✅ Typical Use Case

You consume from topic A → process → produce to topic B.
If processing fails, **both the consume offset and produce messages must rollback**.

🧩 Example (Spring Kafka)

```
@KafkaListener(topics = "input-topic")
@Transactional("kafkaTransactionManager")
public void process(String msg) {
   kafkaTemplate.send("output-topic", msg.toUpperCase());
   // If exception, both offset commit and send rollback
}
```

✅ Spring automatically creates a KafkaTransactionManager bean when producer has a transaction-id-prefix.

💡 **Benefit**

Ensures **“exactly-once”** across producer + consumer.

⚙️ **3. Interceptor in Spring Boot**

✅ **Purpose**

Used to **intercept HTTP requests** before they reach the controller (and/or after response).

🧩 **Example**

```
@Component
public class RequestInterceptor implements HandlerInterceptor {
  @Override
  public boolean preHandle(HttpServletRequest req, HttpServletResponse res, Object handler) {
    System.out.println("Before Controller: " + req.getRequestURI());
    return true;
  }

  @Override
  public void afterCompletion(HttpServletRequest req, HttpServletResponse res, Object handler, Exception ex) {
    System.out.println("After Response");
  }
}
```

Register:

```
@Configuration
public class WebConfig implements WebMvcConfigurer {
  @Override
  public void addInterceptors(InterceptorRegistry registry) {
```

```
        registry.addInterceptor(new RequestInterceptor());
    }
}
```

✅ Used for:

- Logging
- Authentication
- Rate limiting
- Request tracking

💡 **Difference**

Concept	Description
Filter	Works on raw HTTP level (before DispatcherServlet)
Interceptor	Works on Spring MVC level (after DispatcherServlet mapping)

🚀 **4. What Happens in SpringApplication.run()**

✅ **Internals Step-by-Step**

SpringApplication.run(MyApp.class, args);

1. **Create SpringApplication instance**
    - Determines if it's a web app (Reactive, Servlet, or non-web)
2. **Prepare Environment**
    - Loads application.yml, system/env properties
3. **Create ApplicationContext**
    - Loads beans and configurations
4. **Refresh Context**
    - Bean creation, dependency injection
5. **Call CommandLineRunner & ApplicationRunner**
    - Executes startup logic

6. **Start Embedded Server (Tomcat/Jetty)**
    - Listens on configured port
7. **App Ready Event Fired**
    - ApplicationReadyEvent triggers

💡 Order of Events

ApplicationStartingEvent → ApplicationEnvironmentPreparedEvent → ApplicationReadyEvent

---

🔐 5. @PreAuthorize vs @Secured in Spring Security

Annotation	Description	Example	Notes
@PreAuthorize	Uses **SpEL (Spring Expression Language)** for fine-grained access control	@PreAuthorize("hasRole('ADMIN') and #id == authentication.principal.id")	Most flexible
@Secured	Checks only **roles** (simpler)	@Secured("ROLE_ADMIN")	No expression support

---

💡 Best Practice

Use @PreAuthorize for complex rules or dynamic access checks.

Enable:

@EnableGlobalMethodSecurity(prePostEnabled = true, securedEnabled = true)

⚙️ 6. Exception Handling in CompletableFuture

✅ Problem

Async code can throw exceptions — must handle them properly.

🧩 **Example**

```
CompletableFuture.supplyAsync(() -> {
   if (true) throw new RuntimeException("Fail");
   return "OK";
})
.exceptionally(ex -> {
   System.out.println("Handled: " + ex.getMessage());
   return "Default";
});
```

✅ Output:

```
Handled: Fail
```

🧩 **Other Methods**

Method	Description
handle(result, ex)	Runs whether success or failure
whenComplete(result, ex)	Runs after completion, doesn't alter result
exceptionally()	Handles exceptions, returns fallback

💡 **Example**

```
future.handle((res, ex) -> ex == null ? res : "Fallback");
```

⚡ **7. Optimizing Spring Boot Startup Time**

🧠 **Top Causes of Slow Startup**

- Too many auto-configurations
- Large dependency trees
- Heavy database connections
- Classpath scanning overhead

✅ **Best Practices**

Technique	Description
1️⃣ **Lazy Initialization**	Beans loaded on demand
2️⃣ **Profile-specific Beans**	Load only needed configs per environment
3️⃣ **Exclude Unused AutoConfigs**	@SpringBootApplication(exclude = DataSourceAutoConfiguration.class)
4️⃣ **Use AOT (Ahead-of-Time)**	Native image via **Spring Boot 3 / GraalVM**
5️⃣ **Disable Actuator if unused**	Fewer endpoints = faster startup
6️⃣ **Use Connection Pool properly**	Avoid long DB init time (Hikari)
7️⃣ **Parallel Bean Init** (Spring Boot 3.2+)	spring.threads.virtual.enabled=true

---

🧭 **8. Custom Endpoint in Actuator**

You can define **your own Actuator endpoint** to expose metrics or custom health.

🧩 **Example**

```
@Component
@Endpoint(id = "customMetrics")
public class CustomEndpoint {

  @ReadOperation
  public Map<String, Object> metrics() {
    return Map.of("activeUsers", 120, "uptime", "2h 30m");
  }
```

}

✅ Access via:
http://localhost:8080/actuator/customMetrics

💡 **Operation Types**

Type	Annotation
GET	@ReadOperation
POST	@WriteOperation
DELETE	@DeleteOperation

🧩 **9. @ConditionalOnExpression vs @ConditionalOnProperty**

Annotation	Purpose	Example	Use When
@ConditionalOnProperty	Bean loads only if property matches	@ConditionalOnProperty(name="feature.enabled", havingValue="true")	Feature toggles
@ConditionalOnExpression	Bean loads based on **SpEL**	@ConditionalOnExpression("${env} == 'prod'")	Complex logical conditions

---

🧩 **Example**

@Bean

@ConditionalOnProperty(name = "cache.enabled", havingValue = "true")

public CacheManager cacheManager() {

```
    return new ConcurrentMapCacheManager();
}
```

🧱 10. Initialize and Clean Up Data in Spring Boot (@PostConstruct, @PreDestroy)

✅ **Purpose**

Lifecycle annotations for **setup** and **teardown**.

🧩 **Example**

```
@Component
public class DataInitializer {

    @PostConstruct
    public void init() {
        System.out.println("Initializing data...");
        // preload DB, create cache
    }

    @PreDestroy
    public void cleanup() {
        System.out.println("Cleaning up resources...");
        // close connections, clear caches
    }
}
```

✅ Runs:

- @PostConstruct → after bean creation
- @PreDestroy → before context shutdown

💡 **Alternative**

Use Spring events:

@EventListener(ApplicationReadyEvent.class)

public void onStartup() { … }
